





Pointeurs

Définition:

“Le Pointeur est une variable contenant l’adresse d’une autre variable.”

Exemple d’adresse:

- **maVariable** : valeur de la variable
- **&maVariable** : adresse de la variable



pointeurs

Exemple :

```
int *maVariable = 71;
```





<stdio.h>

```
void main(void){
int m=10,n,o;
int *z=&m ;

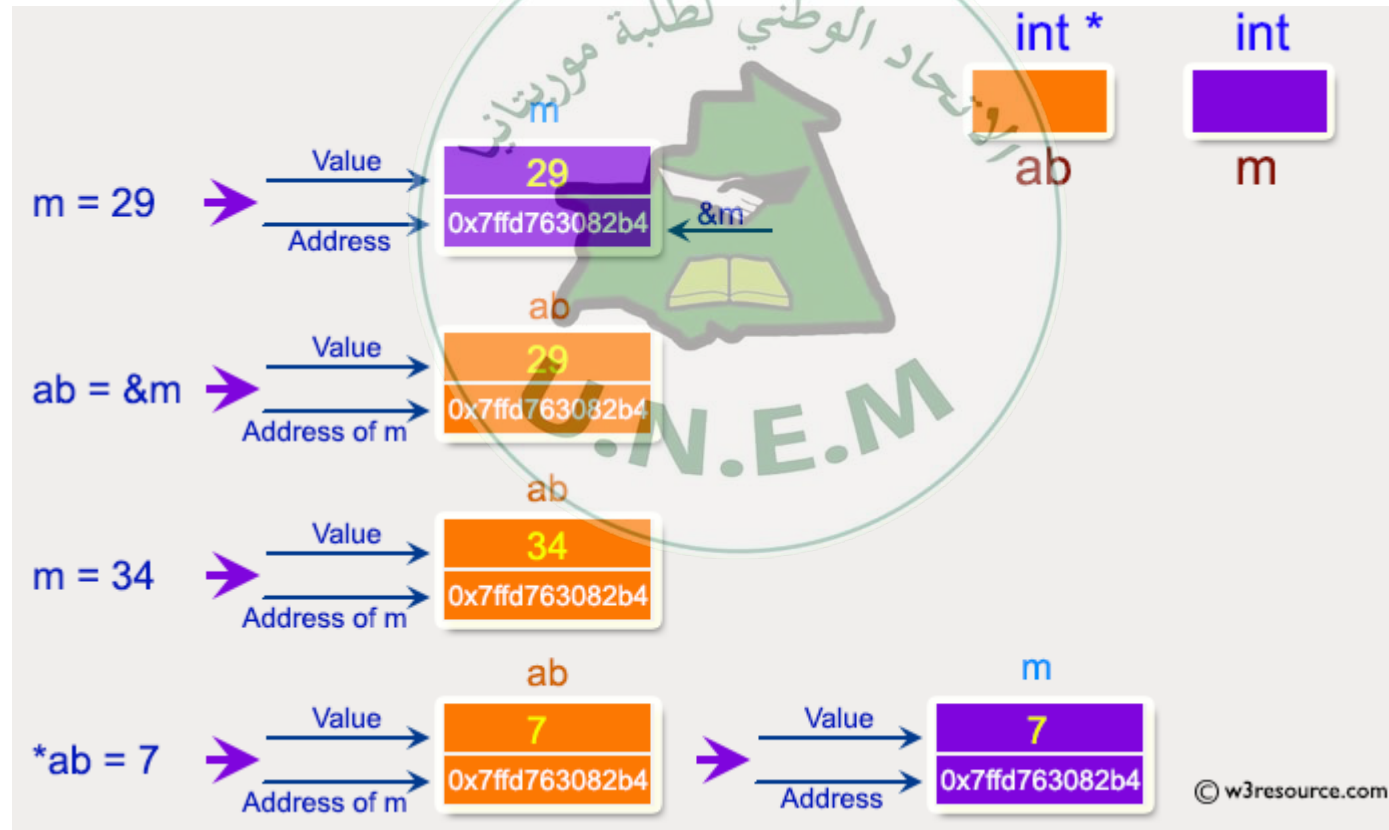
printf("\n\n Pointeur : Afficher une declaration basique d'un pointeur :\n");
printf("-----\n");
printf(" m=10, n et o sont des entiers *z est un pointeur");
printf("\n\n z stocke l'adresse de m = %p\n", z); // z est un pointeur donc %p va printer l'adresse
printf("\n *z stocke la valeur de m = %i\n", *z);
printf("\n &m est l'adresse de m = %p\n", &m); // &m donne l'adresse de la variable entiere m
// donc %p est le specificateur de cette adresse

printf("\n &n stocke l'adresse de n = %p\n", &n);
printf("\n &o stocke l'adresse de o = %p\n", &o);
printf("\n &z stocke l'adresse de z = %p\n\n", &z); // &z donne l'adresse, où le pointeur z est stocké ->
// toujours une adresse -> %p est le bon spécificateur
}
```



pointeurs

Exemple :





```
Void change_nombre(int nb){
    nb = 15;
}

Int main(void){
    int nombre = 0;

    printf("nombre = %d", nombre);
    change_nombre(nombre);
    printf("nombre = %d", nombre);

    return 0;
}
```

```
Void change_nombre(int *nb){
    *nb = 15;
}

Int main(void){
    int nombre = 0;

    printf("nombre = %d", nombre);
    change_nombre(&nombre);
    printf("nombre = %d", nombre);

    return 0;
}
```

```
Int change_nombre(int nb){
    nb = 15;
    return nb;
}

Int main(void){
    int nombre = 0;

    printf("nombre = %d", nombre);
    nombre = change_nombre(nombre);
    printf("nombre = %d", nombre);

    return 0;
}
```



Pointeur - Tableau

- Un tableau de pointeurs est similaire à tout autre tableau en langage C.
- C'est un tableau qui contient de nombreuses variables de pointeur et ces variables de pointeur peuvent stocker les valeurs d'adresse de certaines autres variables ayant le même type de données.



Pointeur - Tableau

- Syntaxe pour déclarer un tableau normal :

`data_type (array_name)[sizeof_array] ;`

Ex : `int arr[10] ;`

- Syntaxe pour déclarer un tableau de pointeurs :

`data_type (*array_name)[sizeof_array] ;`

Ex : `int *ptr[10] ;`



Pointeur - Tableau

```
#include <stdio.h>

int main(){
    // declaration et initialization
    int arr[5] = {3, 5, 7, 9, 11}, i;

    int *ptr = arr;

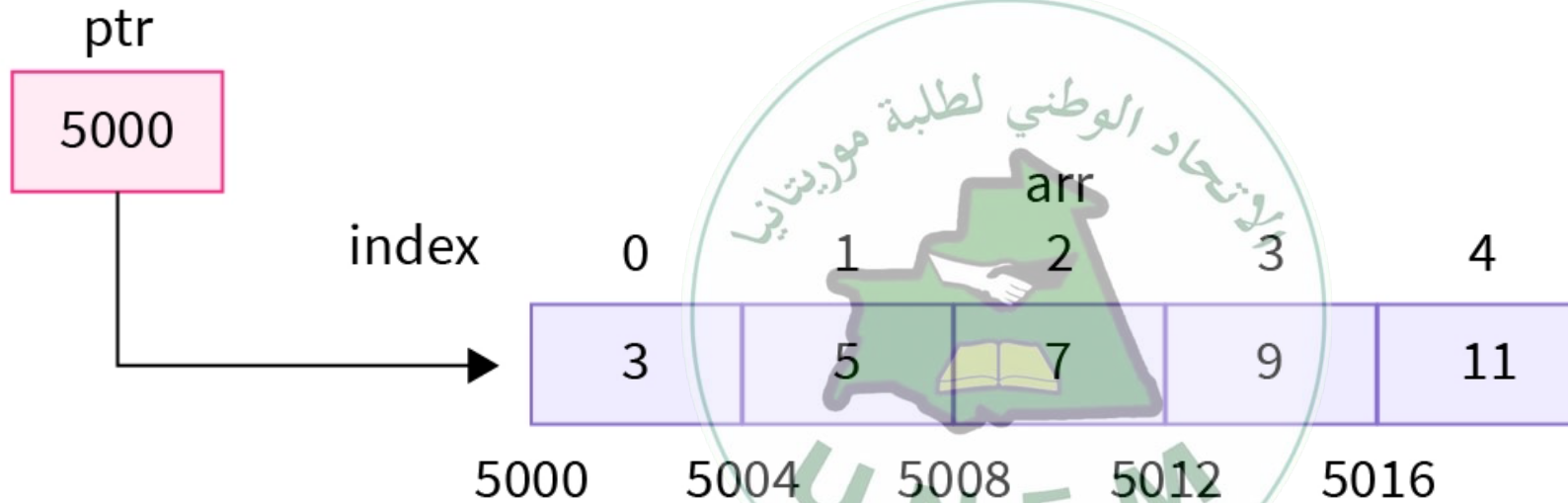
    // affichage des elements du tab en utilisant l'addition sur le pointeur
    for(i = 0; i < 5; i++){
        printf("%d ", *(ptr + i));
    }

    return 0;
}
```

Output : 3 5 7 9 11



Pointeur - Tableau



- $(ptr + i)$ donnera l'adresse des éléments du tableau lorsque la valeur de i change de 0 à 4 comme $address(ptr + i) = address(arr[i])$.
- $*$ est l'opérateur de déréférencement utilisé pour imprimer la valeur à l'adresse fournie. $*(ptr + i)$ imprimera les valeurs du tableau lorsque la valeur de i change.



Pointeur - Tableau

```
#include <stdio.h>
int main(){
// déclaration de tableau et initialisation
int a[5] = {5, 6, 7, 8, 9}, i;

// Valide en cas de tableaux mais non valide en cas de valeurs entières uniques.
int *ptr = a;
// Toutes les représentations impriment l'adresse de base du tableau
printf("ptr : %u, &a[0] : %u, a : %u, &a : %u\n", ptr, &a[0], a, &a);
for(i = 0; i < 5; i++){
// impression des valeurs d'adresse
printf("[index %d] Address : %u\n", i, (ptr + i));
}
printf("\n");
for (i = 0; i < 5; i++) {
// Accéder aux valeurs du tableau via le pointeur
// a[i] = *(a + i) = *(ptr + i) = *(i + a) = a[i]
printf("[index %d] Value : %d %d %d %d\n", i, *(a + i), *(ptr + i), *(i + a), a[i]); }

printf("\n");
// Donne l'adresse du prochain octet après le dernier élément du tableau
printf("&a : %u, &a + 1 : %u\n", &a, &a + 1);

// Donne l'adresse de l'élément suivant
printf("a : %u, a + 1 : %u\n", a, a + 1);

// Donne l'adresse de l'élément suivant
printf("*(a + 1) : %d\n", *(a + 1));

// Donne (valeur à l'index 0) + 1
printf("*a + 1 : %d\n", *a + 1);

// Donne (valeur à l'index 0) / 2, on ne peut pas effectuer *(p / 2) ou *(p * 2)
printf("(*ptr / 2) : %d\n", (*ptr / 2));

return 0;
}
```



pointeur - Tableau

ptr : 6487536, &a[0] : 6487536, a : 6487536, &a : 6487536

[index 0] Address : 6487536

[index 1] Address : 6487540

[index 2] Address : 6487544

[index 3] Address : 6487548

[index 4] Address : 6487552

[index 0] Value : 5 5 5 5

[index 1] Value : 6 6 6 6

[index 2] Value : 7 7 7 7

[index 3] Value : 8 8 8 8

[index 4] Value : 9 9 9 9

&a : 6487536, &a + 1 : 6487556

a : 6487536, a + 1 : 6487540

*(a + 1) : 6

*a + 1 : 6

(*ptr / 2) : 2

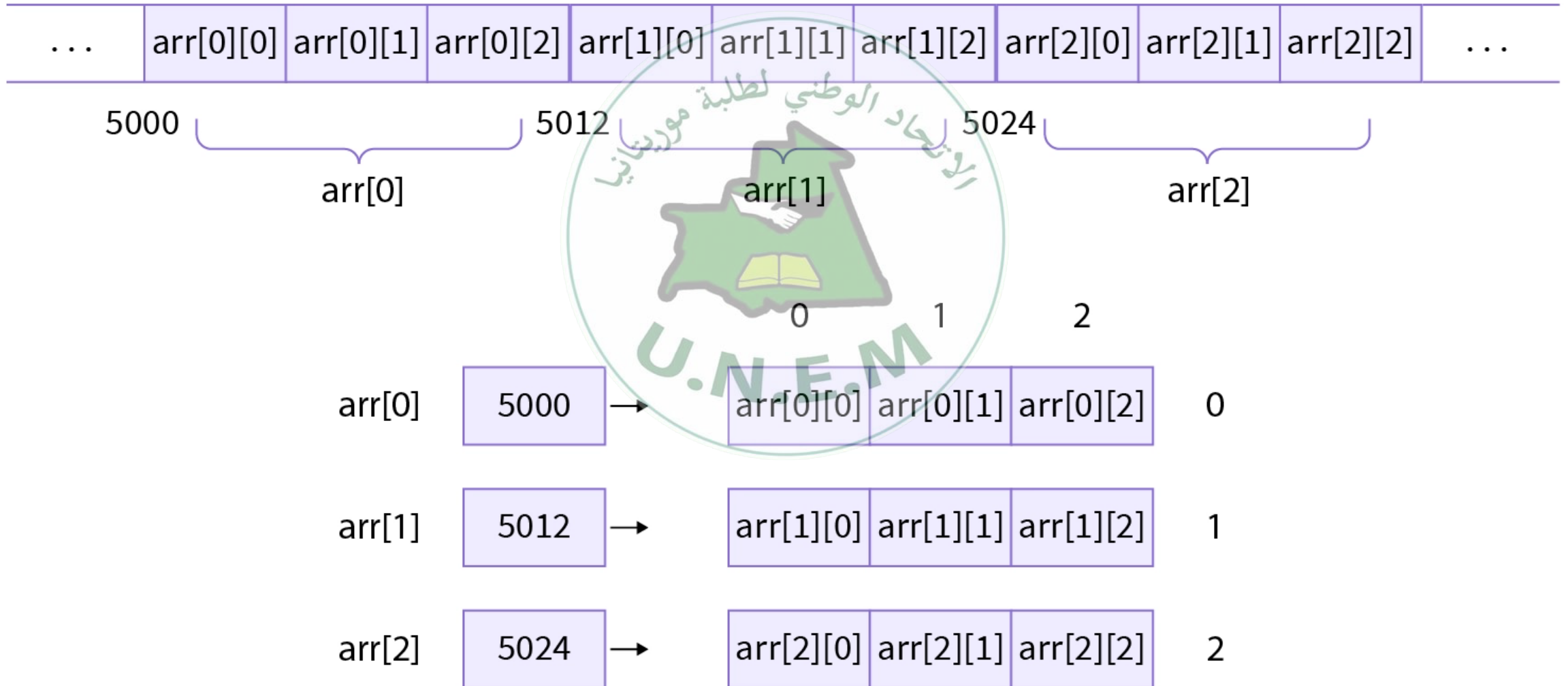


Pointeur – Tableau (2D)

i \ j				
	0	1	2	
0	arr[0][0]	arr[0][1]	arr[0][2]	arr[0]
1	arr[1][0]	arr[1][1]	arr[1][2]	arr[1]
2	arr[2][0]	arr[2][1]	arr[2][2]	arr[2]



Pointeur – Tableau (2D)





Pointeur – Tableau (2D)

- Syntaxe pour représenter les éléments d'un tableau 2D :

$$*(*(\text{arr} + i) + j)$$

Remarque :

$*(*(\text{arr} + i) + j)$ représente l'élément d'un tableau arr à la valeur d'indice de la **i**ème ligne et de la **j**ème colonne ;

cela équivaut à la représentation régulière des éléments de tableau 2D sous la forme **arr[i][j]**.

```
#include <stdio.h>
```

```
int main() {  
    int arr[3][3] = {{2, 4, 6}, {0, 1, 0}, {3, 5, 7}};  
    int i, j;
```

```
    // l'instruction ci-dessous est fausse car  
    // arr renverra l'adresse d'un premier tableau 1-D.  
    // int *ptr = arr;  
    // int*ptr = &arr[0]; est correct ou nous pouvons écrire &arr[1], &arr[2].
```

```
    printf("Addresses : \n");  
    for(i = 0; i < 3; i++){  
        for(j = 0; j < 3; j++){  
            printf("%u[%d%d] ", (*(arr + i) + j), i, j);  
        }  
        printf("\n");  
    }  
    printf("Values : \n");  
    for(i = 0; i < 3; i++){  
        for(j = 0; j < 3; j++){  
            printf("%d[%d%d] ", (*(arr + i) + j), i, j);  
        }  
        printf("\n");  
    }  
    return 0;  
}
```

```
// Output
```

Addresses :

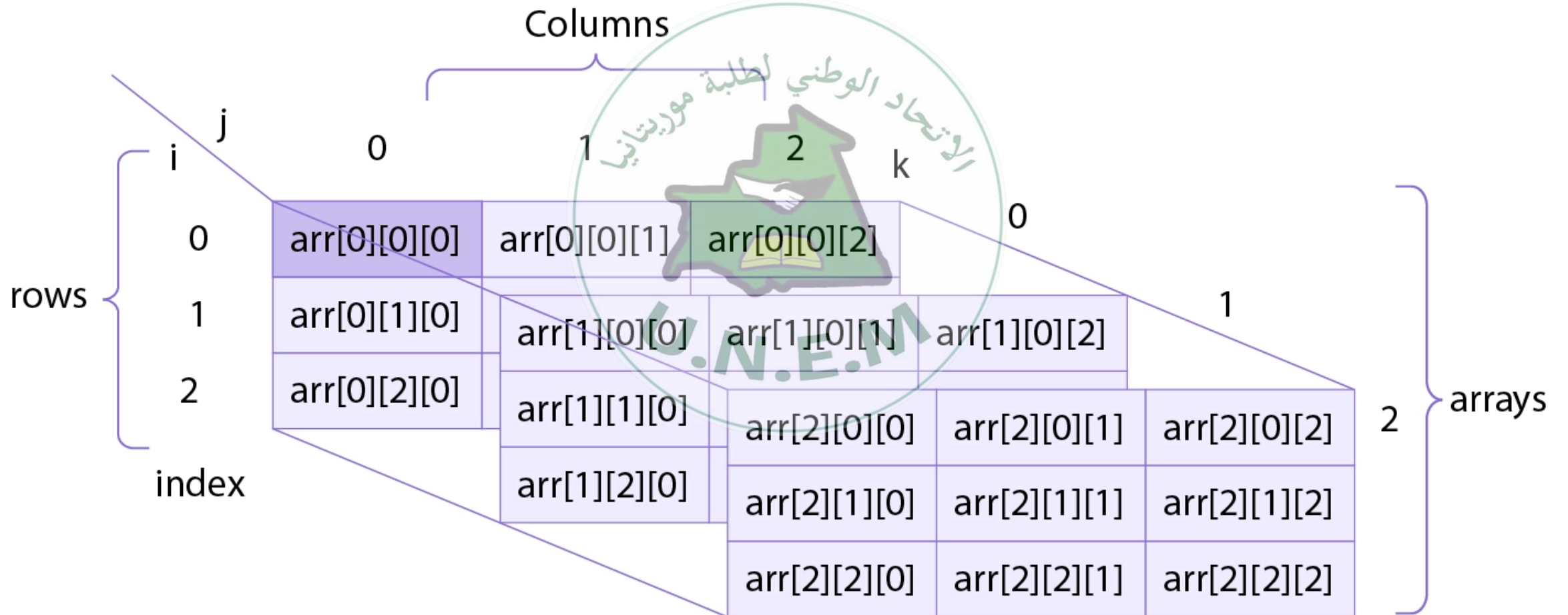
```
6487536[00] 6487540[01] 6487544[02]  
6487548[10] 6487552[11] 6487556[12]  
6487560[20] 6487564[21] 6487568[22]
```

Values :

```
2[00] 4[01] 6[02]  
0[10] 1[11] 0[12]  
3[20] 5[21] 7[22]
```




Pointeur – Tableau 3D





Pointeur – Tableau 3D

Syntaxe pour représenter les éléments d'un tableau 3D :

$*(*(*(arr + i) + j) + k)$

Remarque :

$*(*(*(arr + i) + j) + k)$ représente l'élément d'un tableau arr à la valeur d'index de la **i**ème ligne et de la **j**ème colonne du **k**ème tableau dans le tableau arr ; cela équivaut à la représentation régulière des éléments de tableau 3D sous la forme **arr[i][j][k]**.



```
#include <stdio.h>
```

```
int main(){
```

```
    int arr[3][3][3] = {{1, 2, 3, 4, 5, 6, 7, 8, 9},
```

```
    {2, 4, 6, 8, 10, 12, 14, 16, 18}, {3, 5, 7, 9, 11, 13, 15, 17, 19}};
```

```
    int i, j, k;
```

```
    // l'instruction ci-dessous est fausse car arr renverra l'adresse d'un premier tableau 1-D.
```

```
    // int *ptr = arr; entier *ptr = &arr[0][0] ; est correct ou nous pouvons écrire &arr[1][0], &arr[2][0].
```

```
    printf("Addresses : \n");
```

```
    for(i = 0; i < 3; i++){
```

```
        for(j = 0; j < 3; j++){
```

```
            for(k = 0; k < 3; k++){
```

```
                printf("%u[%d%d%d] ", (*(arr + i) + j) + k, i, j, k);
```

```
            } printf("\n");
```

```
        } printf("\n");
```

```
    } printf("Values : \n");
```

```
    for(i = 0; i < 3; i++){
```

```
        for(j = 0; j < 3; j++){
```

```
            for(k = 0; k < 3; k++){
```

```
                printf("%d[%d%d%d] ", (*(arr + i) + j) + k, i, j, k);
```

```
            } printf("\n");
```

```
        } printf("\n");
```

```
    } return 0;
```

```
}
```



Addresses :

6487456[000] 6487460[001] 6487464[002]
6487468[010] 6487472[011] 6487476[012]
6487480[020] 6487484[021] 6487488[022]

6487492[100] 6487496[101] 6487500[102]
6487504[110] 6487508[111] 6487512[112]
6487516[120] 6487520[121] 6487524[122]

6487528[200] 6487532[201] 6487536[202]
6487540[210] 6487544[211] 6487548[212]
6487552[220] 6487556[221] 6487560[222]

Values :

1[000] 2[001] 3[002]
4[010] 5[011] 6[012]
7[020] 8[021] 9[022]

2[100] 4[101] 6[102]
8[110] 10[111] 12[112]
14[120] 16[121] 18[122]

3[200] 5[201] 7[202]
9[210] 11[211] 13[212]
15[220] 17[221] 19[222]



Pointeur - String

- String est un type de données qui stocke la séquence de caractères dans un tableau.
- Une chaîne en C se termine toujours par un caractère nul (`\0`), qui indique la fin de la chaîne.
- Le pointeur vers une chaîne en C peut être utilisé pour pointer vers l'adresse de début du tableau, le premier caractère du tableau.
- Ces pointeurs peuvent être déréférencés à l'aide de l'opérateur astérisque `*` pour identifier le caractère stocké à l'emplacement.
- Les tableaux 2D et les variables de pointeur peuvent tous deux être utilisés pour stocker plusieurs chaînes.



Pointeur - String

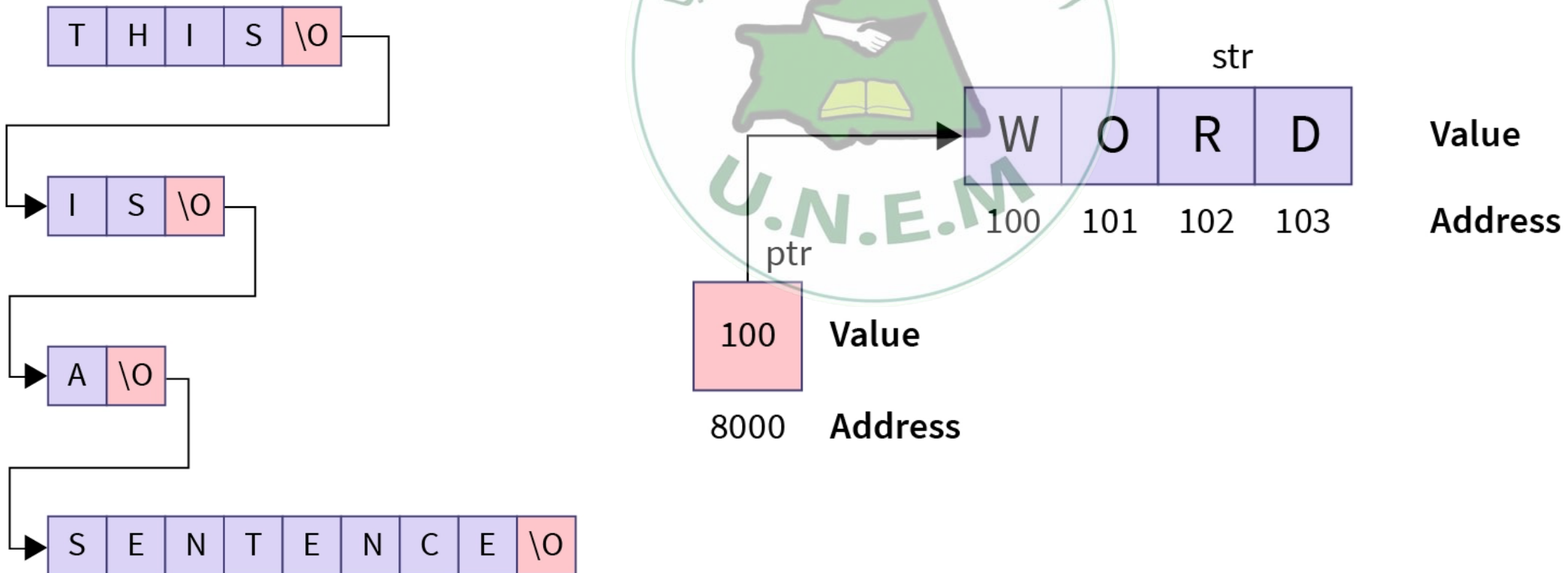
String -> "INTRODUCTION"

I	N	T	R	O	D	U	C	T	I	O	N
---	---	---	---	---	---	---	---	---	---	---	---



Pointeur - String

- Une chaîne se termine toujours par un caractère nul (`\0`), qui indique la fin d'une chaîne. Comme une chaîne est un groupe de caractères de la même manière, nous pouvons également stocker une phrase en C sous la forme d'un groupe de chaînes, comme indiqué dans la figure mentionnée ci-dessous.





Pointeur - String

```
char arr[] = "Hello";  
// pointant le pointeur ptr vers l'adresse de départ  
// du tableau arr  
char *ptr = arr;
```

//Output

```
printf("%c ", *ptr);  
printf("%c ", *(ptr + 1)); // e  
printf("%c ", *(ptr + 2)); // l  
printf("%c ", *(ptr + 3)); // l  
printf("%c ", *(ptr + 4)); // o
```